ADR-010: 階層的ナレッジ圧縮設計（長期運用スケーラビリティ）

ステータス

承認 (2026-03-20) — Phase 2で実装。今から設計に組み込む。

日付

2026-03-20

コンテキスト

現在の設計ではナレッジが増えるほどQ&Aコストとlatencyが線形増加する。

試算:
- 建設会社が2年使い込む → knowledge_items ≈ 5,000件
- Q&A 1回の検索: top-10取得 → embedding + ANN検索は問題ない
- しかしRAG改善後: 再ランキング対象が毎回5,000件から選択 → 問題なし（ANNで絞るため）
- **本当の問題**: ナレッジの陳腐化・矛盾・重複の累積
- 「足場単価¥1,200/m²」と「足場単価¥1,450/m²（2025年改定）」が両方存在
- どちらが正しいかLLMが判断できない → 回答品質低下

また、ゲノムテンプレートが静的JSONのままでは:
- 100社の建設会社が使うと「この会社ではコンクリート打設は¥18,000/m³が標準」という知識が蓄積されるが、テンプレートには反映されない
- 新規テナントが入るたびに全員が同じ基本ナレッジを手動入力する

検討した選択肢

選択肢A: そのまま（知識増加に対応しない）

- デメリット: 2年後に回答品質が劣化。「使い込むほど良くなる」逆になる

選択肢B: 定期クリーンアップ（管理者が手動で削除）

- デメリット: SMBに管理コストをかけさせるとチャーン原因になる

選択肢C: 2層ナレッジ構造（採用・Phase 2実装）

- レイヤー1（要約層）: 同カテゴリのナレッジを定期的にLLMで要約・統合
- レイヤー2（詳細層）: 個別のナレッジアイテム（現在のknowledge_items）
- 検索時: まず要約層でざっくり関連カテゴリを絞り、詳細層から具体的な情報を取得
- メリット: 矛盾・重複を自動統合。新規ユーザーは要約層から即価値を得られる

選択肢D: ベクトルDB専用サービス（Pinecone/Weaviate等）に移行

- デメリット: pgvectorで現状は十分。マルチテナントのRLSが複雑化する

決定

選択肢CをPhase 2で実装する。Phase 1では設計だけ決めておく。

実装設計（Phase 2）

DBスキーマ追加

```
-- ナレッジ要約テーブル
CREATE TABLE knowledge_summaries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL,
    category TEXT NOT NULL,          -- 例: "積算単価", "工程管理"
    summary_text TEXT NOT NULL,      -- カテゴリのナレッジを統合した要約
    embedding VECTOR(768),
    source_item_ids UUID[],          -- どのknowledge_itemsから生成したか
    version INT NOT NULL DEFAULT 1,
    generated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);
```

自動要約バッチ（週次）

```
async def compress_knowledge(company_id: str, category: str):
    """同カテゴリのナレッジ群をLLMで統合要約する"""
    items = await get_active_knowledge_by_category(company_id, category)
    if len(items) < 5:
        return  # まだ少なすぎる

    summary = await llm.generate(
        "以下のナレッジを統合して、最新情報を優先した要約を作成してください...",
        tier=ModelTier.STANDARD
    )
    await upsert_knowledge_summary(company_id, category, summary)
```

ゲノムテンプレートへのフィードバック（Phase 3）

```
100テナントの要約データ → 匿名化 → 業界標準テンプレートに反映
→ 新規テナントが初日から「業界平均値入りのテンプレート」を使える
```

Phase 1での準備（今すぐやること）

1. `knowledge_items` に `category` カラムが既にある ✅ → 要約のキーになる
2. `is_active` フラグが既にある ✅ → 要約で置き換えた古いアイテムを非アクティブ化できる
3. **今やること**: `knowledge_items` テーブルに `half_life_days` カラムを追加（ADRで切り落としていた機能）
   - Phase 1: 手動更新でTTLを設定
   - Phase 2: 自動要約時に古いアイテムのhalf_lifeを短くして自然に消える設計

影響

- Phase 2の実装コストが明確化: DB migration + バッチ処理 + テスト
- Phase 1では影響なし（設計決定のみ）
- ゲノムテンプレート静的JSONはPhase 3まで維持（正しい判断）

関連

- ADR-006: MVP scope reduction（half-life modelをPhase 2に先送りした決定）
- ADR-007: RAGパイプライン改善（enhanced_searchの改善先）
- `shachotwo-app/brain/genome/` （テンプレートフィードバック先）